Alexandria University
Faculty of Engineering
Computer and Communication Engineering
Specialized Scientific Programs



Computer and Communications
Programming II
Fall 2026

Lab 5

MARS-OPS

Autonomous Mars Colony Management & Emergency Simulator

Lab Mission

Imagine humans have built a colony on Mars, and you need to create a Java program that manages the colony, its resources, its people, and emergencies.



Dr Mervat Mikhail
Eng. Ahmed Ashraf
Eng. Kyrollos Youssef
Eng. Mahinour Emad
Eng. Mariam Hassan

Dr Ahmed Elsayed
Eng. Tarek Salah
Eng. Seif Eldin Mahmoud
Eng. Yara Hazem
Eng. Abdelrahman Ayoub

1. Project Objectives

By completing this assignment, you should demonstrate that you can:

1. Analyze a real-world problem and identify appropriate objects and classes.
2. Apply the four fundamental Object-Oriented Programming (OOP) concepts: encapsulation, abstraction, inheritance, and polymorphism.
3. Model relationships between objects.
4. Distinguish between inheritance, association, aggregation, and composition.
5. Design a system using UML before implementation.
6. Translate a UML model into Java.
7. Design objects that collaborate rather than placing all logic inside one class.
8. Handle changing system state through a simulation.
9. Explain and defend your design decisions.

Dr Mervat Mikhail
Eng. Ahmed Ashraf
Eng. Kyrollos Youssef
Eng. Mahinour Emad
Eng. Mariam Hassan

Dr Ahmed Elsayed
Eng. Tarek Salah
Eng. Seif Eldin Mahmoud
Eng. Yara Hazem
Eng. Abdelrahman Ayoub

2. Project Overview

Humanity has established a small research colony on Mars. The colony operates with limited resources and a mixture of human personnel, autonomous robots, vehicles, habitats, laboratories, and specialized equipment.

Your team has been asked to develop the **MARS-OPS** software system, responsible for managing the colony and coordinating missions and emergency situations.

The system must be capable of:

- Managing colony personnel and robotic units.
- Managing habitats and facilities and tracking resources.
- Assigning missions and simulating the passage of time.
- Determining whether available entities can perform missions.
- Handling equipment failures, responding to emergencies and updating the state of the colony after operations.
- Producing a colony status report.

The challenge is not simply to store information about these objects. The system must model how these objects interact with each other.

This specification deliberately does not tell you which classes to create. Deciding what the objects are, what each is responsible for, and how they relate is a central part of the assignment.

3. The Mars Colony

3.1 Human Personnel

Examples include scientists, engineers, and medical officers. Each person has characteristics such as:

- Name.
- Identification number.
- Health status.
- Current location.
- Skills.
- Fatigue level.
- Availability.

Different personnel types have different capabilities:

Personnel Type	**Primary Responsibility**
Scientist	Performs scientific research missions.
Engineer	Repairs damaged equipment.
Medical Officer	Responds to medical emergencies.

3.2 Robotic Units

The colony also operates autonomous robots, including exploration rovers, repair robots, and drones.

Different robots have different capabilities.

Robot Type	Capabilities
Exploration Rover	Travels across the Martian surface, carries equipment, and performs exploration.
Repair Robot	Repairs damaged equipment and operates inside facilities.
Drone	Performs aerial reconnaissance and operates in areas inaccessible to ground vehicles.

Robots have operational states such as:

- Battery level.
- Health or damage status.
- Current location.
- Availability.

4. Colony Facilities

The colony contains multiple facilities, for example:

- Habitats.
- Laboratories.
- Storage facilities.
- Power stations.
- Communication stations.

Facilities may have:

- A location.
- Capacity.
- Operational status.
- Resources.
- Equipment.

Some facilities contain other objects. You must determine the appropriate ownership relationships between these objects.

5. Resources

The colony has limited resources, including:

- Oxygen.
- Water.
- Food.
- Fuel.
- Battery power.
- Medical supplies.
- Spare parts.

Resources must be represented in the system, and their quantities must be tracked. Operations may consume resources.

Examples:

- A repair mission might consume 2 spare parts and 5 battery units.
- A medical emergency might consume medical supplies and oxygen.

The colony must never allow an operation to consume more resources than are available.

5.1 Where Resources Are Stored

Resources are stored in facilities, such as oxygen in Habitat A and oxygen in the Storage Facility, but they are managed at the colony level.

The following rules must be implemented:

1. **Colony-wide totals:** The colony's total quantity of a resource is the sum of the quantities held across all currently operational facilities. Resources held in a destroyed or unavailable facility do not count.
2. **Multiple sources:** A mission or operation may draw a required resource from more than one facility. For example, if Habitat A holds 300 units of oxygen and the Storage Facility holds 200 units, an operation requiring 450 units is valid, but one requiring 600 units is not.
3. **All-or-nothing consumption:** If the full required quantity cannot be supplied, no resources are consumed, and the operation fails with a clear reason.
4. **Local emergencies:** An oxygen leak reduces the oxygen stored in the affected habitat only, thereby reducing the colony-wide total.
5. **Colony status report:** The report must display colony-wide resource totals.
6. **Resource allocation policy:** Choosing which facility supplies resources first is a design decision. Document your chosen policy in the README assumptions.

6. Missions: The colony must support different types of missions.

Mission Type	Description	Requirements
Exploration	Explores a specified region.	Suitable vehicle, required scientific skill, sufficient battery, and required equipment.
Repair	Repairs of a damaged facility or piece of equipment.	Engineer, repair robot, and spare parts.
Research	A scientist performs an experiment in a laboratory.	Scientists with a particular skill, laboratory, scientific equipment, and sufficient resources.
Rescue	Locates and recovers a stranded entity.	Appropriate vehicle, available personnel, and communication capability.

6.1 Capability Requirements (Important)

A mission must not simply check the concrete type of object.

The design should allow the system to ask an entity whether it is capable of performing a particular mission. Conceptually:

```
if (agent.canPerform(mission)) {
    mission.assign(agent);
}
```

The exact design is left to you.

6.2 One Active Mission at a Time

The colony may have only one active mission at a time.

A mission is submitted to the colony together with the agents it requires.

The following rules apply:

1. If another mission is active, the new mission immediately becomes FAILED.
2. If an agent lacks the required capability, the mission immediately becomes FAILED.
3. If the required resources are insufficient, the mission immediately becomes FAILED.
4. Any rejected mission must have a clear reason for failure and must never become active.
5. If the submission is accepted, the mission becomes ASSIGNED and remains the active mission until it completes or fails.
6. Failed missions are not retried. A new mission must be created to try again.
7. A mission may require several agents. All assigned agents become available again when the mission ends.

Emergency responses are not subject to the one-active-mission limit. However, responders must be available, meaning they must not be part of the active mission.

6.3 Mission Ownership

Missions are received and tracked by the colony as a whole.

- Only one mission can be active at a time.
- Agent availability is checked colony-wide.
- A mission may refer to one or more facilities as its target or location, such as the facility to repair or the laboratory used for research.
- Some missions, such as exploration, may not refer to any facility.
- Facilities do not own or schedule missions.
- Mission resource requirements are checked against the colony-wide totals described in Section 5.

7. Mission Execution

Each mission has a lifecycle. You must decide how mission states are represented and controlled. Each mission has a duration measured in time units. For example, an exploration mission may take 3-time units.

The following rules apply:

1. A mission starts counting its duration when it becomes IN_PROGRESS.
2. Each time unit advances the mission by one unit.
3. When the required duration is reached, the mission becomes COMPLETED.
4. When the mission completes or fails, its assigned agents become available again.
5. A mission may fail because of conditions such as:
    - Insufficient resources.
    - Equipment failure.
    - An unsuitable agent.
    - Environmental conditions.
    - Insufficient battery.
    - Unavailable personnel.

Your design must determine how mission states are represented and controlled.

8. Mars Environment

The environment affects colony operations. The colony may experience:

- Normal conditions.
- Dust storms.
- Communication disruption.

8.1 Dust Storm

- Surface exploration missions cannot begin.
- Active rover missions fail.

8.2 Communication Disruption

- Remote drones cannot be controlled.
- Certain missions cannot be initiated.
- Communication-dependent operations may fail.

The design should allow environmental conditions to affect different operations appropriately.

Documentation requirement: Explain in the README what happens to missions already in progress when environmental conditions change.

Dr Mervat Mikhail
Eng. Ahmed Ashraf
Eng. Kyrollos Youssef
Eng. Mahinour Emad
Eng. Mariam Hassan

Dr Ahmed Elsayed
Eng. Tarek Salah
Eng. Seif Eldin Mahmoud
Eng. Yara Hazem
Eng. Abdelrahman Ayoub

9. Emergencies

Emergency handling is one of the most important parts of simulation.

The colony may experience several types of emergencies, including oxygen leaks, fires, rover breakdowns, and medical emergencies.

9.1 Locations and Responders

The location is either a facility or the Martian surface.

The following rules apply:

1. Every agent has a current location.
2. A rover on an exploration mission is on the surface.
3. A rover that breaks down remains where it broke down.
4. Location does not restrict who can respond. Any available agent may be dispatched, and dispatching moves the agent to the affected facility.
5. A responder is appropriate if it reports that it can handle the emergency type. Ask the agent; do not check its concrete type.
6. A responder is available if it is not part of an active mission and is in usable condition.
7. Each emergency type requires specific responders.

For example, an oxygen leak requires at least one human and one robotic responder capable of performing repairs, such as an engineer and a repair robot.

You must decide the requirements for the other emergencies and document them in the README.

Emergencies do not interrupt active missions.

If the required responders are unavailable:

- The emergency is marked UNRESOLVED.
- A clear reason is recorded.
- No response resources, such as spare parts or battery power, are consumed.
- The unresolved emergency is not retried automatically.

9.2 Oxygen Leak

An oxygen leak occurs inside a habitat.

The system should:

1. Handle the emergency when manually triggered.
2. Identify the affected habitat.
3. Reduce the oxygen available in the affected habitat.
4. Determine appropriate responders.
5. Dispatch available responders.
6. Consume the required resources.
7. Repair the leak if possible.
8. Update the state of the habitat.

Oxygen Cost of a Leak

The amount of oxygen lost to a leak is defined by your scenario.

The following rules apply:

- When the leak is detected, the specified amount of oxygen is deducted from the affected habitat.
- If the leak is repaired, this is the total oxygen loss.

- If the emergency ends unresolved, the same amount is deducted again.
- Therefore, an unresolved leak costs twice as much oxygen as a repaired leak.
- A habitat's oxygen level must never fall below zero.
- This is a one-time deduction. Leaks do not drain oxygen continuously over time.

9.3 Fire

A laboratory catches fire.

Possible consequences include:

- The laboratory becomes unavailable.
- Equipment becomes damaged.
- Oxygen consumption increases.
- Fire-response personnel are required.
- Medical assistance may be required for all personnel present in the laboratory.

Your design must determine how these consequences are represented and applied.

9.4 Rover Breakdown

A rover becomes damaged during an exploration mission.

The system must update:

- Rover status.
- Mission status.
- Rover location.
- Required repair resources.

A suitable repair operation may then be created.

9.5 Medical Emergency

A crew member requires urgent medical attention.

The system must determine:

- Whether an appropriate medical officer is available.
- Whether the required medical supplies exist.
- Whether the relevant facility is operational.

After successful treatment, the patient's health should increase.

10. Simulation and Time Management

The system must support the concept of time.

The simulation should be able to advance by one time unit, for example:

simulation.advanceTime(1);

An equivalent design is also acceptable.

Time advancement may cause:

- Resources to change, Missions to progress, Batteries to decrease, Fatigue to increase, Environmental conditions to change, and Emergencies to occur.

The exact implementation is up to you.

11. Object Relationships and UML Design

The project must demonstrate meaningful object relationships and include five UML diagram types: **Class Diagram, Object Diagram, State Machine Diagram, Sequence Diagram, and Use Case Diagram**.

All diagrams must be created using a UML or diagramming tool (For example: draw.io) and must be consistent with the final Java implementation.

11.1 Object Relationships

The system must demonstrate the following UML relationship types:

1. **Association:** Represents objects that interact with or use one another.



MARS-OPS example: An engineer works on a repair mission.

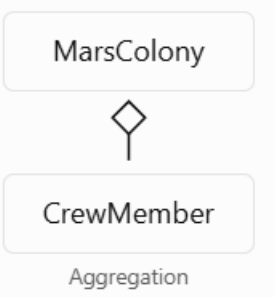
```
class Engineer {
    void repair(Mission mission) {
        System.out.println("Engineer is repairing.");
    }
}
```

Here, the Engineer interacts with a Mission. The mission can exist without that particular engineer, and the engineer can exist without that mission.

UML representation: A solid line connects the two classes.

2. **Aggregation:** Represents a whole-part relationship in which the parts can exist independently of the whole.





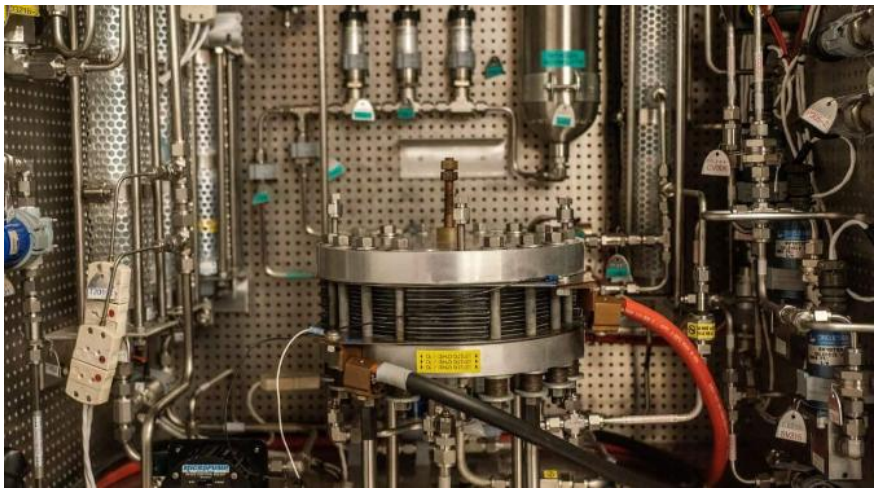
MARS-OPS example: A MarsColony has several CrewMember objects.

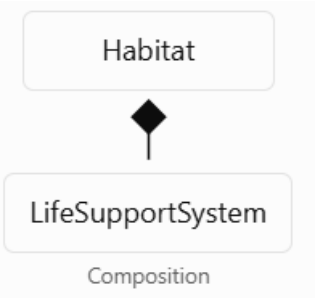
```
class MarsColony {
    private List<CrewMember> crew;
    MarsColony(List<CrewMember> crew) {
         this.crew = crew;
        }
}
```

If the colony is shut down or removed, the crew members can still exist as objects and potentially be assigned elsewhere.

UML representation: A solid line with a hollow diamond at the grouping end.

3. **Composition:** Represents a strong ownership relationship in which a part's lifecycle is tied to the whole.





MARS-OPS example: A Habitat owns its internal LifeSupportSystem.

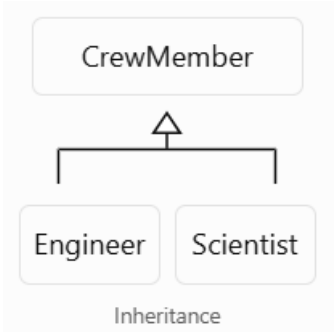
```
class LifeSupportSystem {// Manages oxygen and air filtration}
class Habitat {
    private final LifeSupportSystem lifeSupport;
    Habitat() {
        this.lifeSupport = new LifeSupportSystem();
     }
}
```

In this example, each habitat creates and owns its own life-support system. If the habitat is destroyed, its internal life-support system is considered part of that habitat and is removed with it in the model.

UML representation: A solid line with a filled diamond at the owner's end.

4. **Inheritance:** Represents an "is-a" relationship between a parent class and its specialized subclasses.





MARS-OPS example: An Engineer is a type of CrewMember, just as a Scientist is a type of CrewMember.

```
class CrewMember {

   protected String name;

   void performDuty() {

      System.out.println("Performing a duty.");

   }

}
```

```
class Engineer extends CrewMember {
  @Override
  void performDuty() {
    System.out.println("Repairing equipment.");
  }
}
class Scientist extends CrewMember {
  @Override
  void performDuty() {
    System.out.println("Conducting research.");
  }
}
```

Both subclasses inherit from CrewMember, but they can perform their duties differently. This also demonstrates **polymorphism**.

UML representation: A solid line with a hollow triangle pointing toward the parent class.

You must explain why each relationship was chosen and how it is represented in both the UML diagrams and the Java implementation.

11.2 Class Diagram (Required)

The class diagram must represent the static structure of the MARS-OPS system.

It must include:

- Major classes, interfaces, abstract classes, and enumerations, where applicable.
- Attributes, data types, methods, parameters, and return types.
- Visibility of important attributes and methods.
- Relationships between classes, including association, aggregation, composition, and inheritance.
- Multiplicities where relevant, such as one colony managing multiple crew members.
- The responsibilities of the major classes.

The diagram should cover the main components of the system, including personnel, robotic units, facilities, resources, missions, emergencies, and simulation management.

It must clearly demonstrate the four required relationship types described in Section 11.1.

11.3 Object Diagram (Required)

The object diagram must show a snapshot of the system at a specific moment during execution.

It must include:

- Representative object instances created from the classes in the class diagram.
- Object names and class names, such as rover1:ExplorationRover.
- Relevant attribute values, such as battery level, oxygen quantity, and mission state.
- Links between objects that correspond to the relationships defined in the class diagram.

Required demonstration: Show a colony containing at least one crew member, one robotic unit, one facility, and one mission. The snapshot must represent a consistent system state, such as immediately before an exploration mission or after an emergency response.

11.4 State Machine Diagram (Required)

The state machine diagram must describe the lifecycle of a mission.

It must include:

- All relevant mission states.
- Transitions between states.
- Events and conditions that trigger transitions.
- Guard conditions where appropriate.
- Initial and final states.
- Failure transitions and their causes.

The diagram must represent the mission lifecycle, including:

- ASSIGNED
- IN_PROGRESS
- COMPLETED
- FAILED

It must show that an accepted mission progresses from ASSIGNED to IN_PROGRESS, then to COMPLETED when its duration is reached, unless a failure occurs.

It must also show that:

- A rejected mission immediately becomes FAILED and never becomes active.
- A mission in progress may fail because of conditions such as equipment failure, environmental restrictions, or insufficient battery.
- Failed missions cannot be retried; a new mission must be created for another attempt.

11.5 Sequence Diagrams (Required)

Sequence diagrams must illustrate how objects collaborate to perform operations over time.

Provide sequence diagrams for the following three operations:

A. Mission Submission and Execution, Show:

1. Submitting a mission to the colony.
2. Checking whether another mission is active.
3. Checking agent capabilities and availability.
4. Checking resource availability.
5. Accepting or rejecting the mission.
6. Updating the mission state.
7. Advancing simulation time and progressing the mission.
8. Completing or failing the mission and releasing its agents.

Include an alternative flow for a rejected mission, showing the failure reason and confirming that the mission never becomes active.

B. Oxygen Leak Emergency, Show:

1. Triggering the emergency.

2. Identifying the affected habitat.
3. Deducting the initial oxygen loss.
4. Identifying appropriate responders and checking their availability.
5. Checking required response resources.
6. Dispatching responders and attempting the repair.
7. Applying the additional oxygen loss if the emergency remains unresolved.
8. Updating the habitat and emergency states.
9. Releasing responders when the response ends.

Include alternative flows for unavailable responders and insufficient resources. Follow the specifications rules for resource consumption and oxygen loss.

C. Environmental Restriction

Show how a dust storm prevents an exploration mission from starting.

Include the environmental check, the rejection of the operation, and the resulting failure reason.

Each sequence diagram must identify participating objects, messages exchanged between them, and the order of interactions. Use alternative fragments where appropriate to represent success and failure conditions.

11.6 Use Case Diagram (Required)

The use case diagram must describe the system's main functionality from the perspective of external actors.

It must include:

- The MARS-OPS system boundary.
- Relevant actors.
- Use cases representing the main system operations.
- Associations between actors and use cases.
- Appropriate <<include>> and <<extend>> relationships where justified.

Potential actors include a Colony Operator and an Emergency Operator.

The diagram must cover the following use cases:

- Manage crew members and robotic units.
- Manage facilities and equipment.
- Monitor resources.
- Generate colony status reports.
- Submit, assign, execute, and monitor missions.
- Advance simulation time.

- Monitor environmental conditions.
- Trigger and handle emergencies.
- Inspect mission and emergency outcomes.

Use <<include>> when one use case always requires another use case, and <<extend>> when additional behavior occurs conditionally. These relationships must reflect actual system behavior.

11.7 UML Diagram Consistency and Submission

All five UML diagrams must be consistent with one another and accurately reflect the final Java implementation.

- The class diagram must match the implemented classes, methods, attributes, and relationships.
- The object diagram must contain valid instances of the classes.
- The state machine diagram must match the implemented mission state transitions.
- The sequence diagrams must reflect the actual interactions and responsibilities of the objects.
- The use case diagram must represent functionality supported by the application.

12. Required Demonstration Scenarios

The program must demonstrate at least the following five scenarios.

Scenario 1: Successful Exploration

Create a scientist, an exploration rover, appropriate equipment, and an exploration mission. The mission should execute successfully.

Show the system state before and after execution.

Scenario 2: Mission Rejection

Submit a mission with an unsuitable agent.

The mission must fail immediately with a meaningful reason and must never become active. Example output: Mission cannot be assigned.

Reason: Agent does not possess the required capability.

Scenario 3: Resource Shortage

Attempt to execute a mission when the required resources are insufficient.

The system must reject the operation without consuming unavailable resources or allowing resource quantities to become negative.

The failure must be reported clearly.

Scenario 4: Emergency Handling

Simulate an oxygen leak.

The system must:

1. Detect the emergency.
2. Locate the affected facility.
3. Identify suitable responders.
4. Perform the response if possible.
5. Consume the required resources.
6. Update the facility state.

Scenario 5: Environmental Effect

Introduce an environmental condition, such as a dust storm, and demonstrate that it affects at least one operation.

Example output:

Exploration Mission cannot start.

Reason: Surface operations are not allowed during a dust storm.

13. Deliverables

The completed MARS-OPS project must include the following deliverables.

- **Java Programs (All Java Source Code)**
- **UML Design**
- **README and Design Documentation**
  The README must include:
  - A description of the system and its main components.
  - An explanation of important design decisions and assumptions.
  - Documentation of resource allocation rules.
  - Documentation of emergency rules.
  - Documentation of environmental effects.
  - Documentation of mission states.
  - Explanations of other implementation decisions left open by the specification.
  - Instructions for compiling and running the application.
  - Instructions for executing the required demonstration scenarios.
- **Testing (THE MAIN CLASS)**

14. Notes

- Design and implement a working simulation that demonstrates how a Mars colony can manage its personnel, robots, facilities, resources, missions, environmental conditions, and emergencies through a well-structured
- Submit all diagrams as clearly readable images or PDF files or include them in the project documentation. Include editable source files when supported by the chosen UML tool.
- You must be prepared to explain the purpose of each diagram, justify your design decisions, and demonstrate how the UML design corresponds to the Java implementation.

15. Submission Requirements

Submit one ZIP file

15.1. Folder Structure

- Organize all submission files in a single folder, for example:
  StudentID-Prog-II-Lab5/ (your code files)

15.2. ZIP Submission

- Compress the folder StudentID-Prog-II-Lab5 into a .zip file.
- Verify that your project compiles and runs correctly before submitting.
- The submitted ZIP file must be complete and ready to run.

Policy:

- Late submission is not allowed.
- Copies will be heavily penalized.
- Online submission on time is a mandatory step for your submission.
- You will be penalized if you don't attend the discussion on time (or at all), so stick with your timeslot.
- You should obey the submission format explained above.
- You should work in groups of 4.
- All group members are required to actively participate in the lab assignment and collaborate throughout all stages of development.
- Work must be shared fairly among all group members. Every member is expected to contribute to the design, implementation, testing, and documentation of the project.
- All group members must understand the overall system and be able to explain the design decisions and implementation, including the parts they did not personally implement.
- The project must be the result of genuine teamwork and collaboration, rather than being completed independently by individual members with no meaningful collaboration.
- The use of AI tools in any form to complete or generate project work, as well as any form of cheating, plagiarism, or academic misconduct, is strictly prohibited. Any group found violating these rules will face severe penalties.

Dr Mervat Mikhail
Eng. Ahmed Ashraf
Eng. Kyrollos Youssef
Eng. Mahinour Emad
Eng. Mariam Hassan

Dr Ahmed Elsayed
Eng. Tarek Salah
Eng. Seif Eldin Mahmoud
Eng. Yara Hazem
Eng. Abdelrahman Ayoub